

Aula Detalhada - Ponto Fixo E Ponto Flutuante

Tema do dia: frações em binário, ponto fixo, escala, formato Q, ponto flutuante em nível conceitual, sinal, expoente, mantissa/significando, normalização, precisão e arredondamento

Aula na sequência: 17

Objetivo: entender como números com parte fracionária podem ser representados em sistemas digitais, diferenciar ponto fixo de ponto flutuante e reconhecer os principais erros de interpretação em questões de prova.

1. Onde Esta Aula Entra No Estudo?

Nas aulas anteriores, você estudou:

números inteiros em binário
complemento de 2
soma e overflow
somadores
subtrator
ULA
shifters

Agora vamos responder outra pergunta:

como representar números que não são inteiros?

Exemplos:

3,5
0,25
-2,75
13,125

Existem duas ideias principais:

ponto fixo
ponto flutuante

Ponto fixo é mais simples e mais rígido.

Ponto flutuante é mais flexível e permite representar números muito pequenos e muito grandes, mas traz arredondamento e perda de precisão.

2. Parte Fracionária Em Binário

2.1 Posições antes e depois do ponto

Em decimal, as posições são potências de 10.

Exemplo:

123,45

Significa:

$1*100 + 2*10 + 3*1 + 4*0,1 + 5*0,01$

Em binário, as posições são potências de 2.

Exemplo:

101,101

Antes do ponto:

$1*4 + 0*2 + 1*1$

Depois do ponto:

$1*(1/2) + 0*(1/4) + 1*(1/8)$

Tabela:

Posição	Peso
2^3	8
2^2	4
2^1	2
2^0	1
2^{-1}	$1/2 = 0,5$
2^{-2}	$1/4 = 0,25$
2^{-3}	$1/8 = 0,125$
2^{-4}	$1/16 = 0,0625$

2.2 Exemplo: binário fracionário para decimal

Converta:

101,101_2

Parte inteira:

$$101_2 = 5$$

Parte fracionária:

$$0,101_2 = 1/2 + 0/4 + 1/8$$

$$0,101_2 = 0,5 + 0 + 0,125$$

$$0,101_2 = 0,625$$

Total:

$$101,101_2 = 5,625_{10}$$

2.3 Frações binárias comuns

Vale memorizar:

Binário	Decimal
0,1	0,5
0,01	0,25
0,001	0,125
0,0001	0,0625
0,11	0,75
0,101	0,625
0,111	0,875

Isso ajuda muito em ponto fixo e ponto flutuante.

3. Decimal Fracionário Para Binário

3.1 Parte inteira

A parte inteira você já conhece:

divide por 2
pega os restos
lê de baixo para cima

Exemplo:

$$13_{10} = 1101_2$$

3.2 Parte fracionária

Para a parte depois da vírgula, use multiplicações por 2.

Regra:

multiplique a fração por 2
pegue a parte inteira como próximo bit
continue com a parte fracionária restante

Exemplo:

$$0,625$$

Passos:

Passo	Fração * 2	Bit gerado	Fração restante
1	$0,625 * 2 = 1,25$	1	0,25
2	$0,25 * 2 = 0,5$	0	0,5
3	$0,5 * 2 = 1,0$	1	0

Resultado:

$$0,625_{10} = 0,101_2$$

3.3 Exemplo completo

Converta:

$$13,625_{10}$$

Parte inteira:

$$13_{10} = 1101_2$$

Parte fracionária:

$$0,625_{10} = 0,101_2$$

Resultado:

```
13,625_10 = 1101,101_2
```

3.4 Nem toda fração decimal termina em binário

Assim como:

```
1/3 = 0,33333...
```

não termina em decimal, algumas frações decimais não terminam em binário.

Exemplo famoso:

```
0,1 decimal
```

Em binário, ele vira uma dízima periódica:

```
0,0001100110011...
```

Por isso computadores podem ter pequenos erros com ponto flutuante.

4. Ponto Fixo

4.1 O que é

Ponto fixo é uma representação em que a posição do ponto binário é fixa.

Exemplo:

```
8 bits totais
4 bits para a parte inteira
4 bits para a parte fracionária
```

Formato:

```
IIII.FFFF
```

O ponto não fica armazenado no número.

Ele é combinado pelo formato.

Isso é muito importante:

o hardware guarda bits
o programador/circuito sabe onde o ponto está

4.2 Exemplo simples

Considere o formato:

4 bits inteiros
4 bits fracionários

O padrão armazenado é:

0011 0100

Como o ponto fica depois dos 4 primeiros bits:

0011,0100

Parte inteira:

0011 = 3

Parte fracionária:

$0100 = 0/2 + 1/4 + 0/8 + 0/16$
 $0100 = 0,25$

Valor:

3,25

4.3 Escala

Escala é a ideia mais importante de ponto fixo.

Pense assim:

o circuito guarda um inteiro
mas você interpreta esse inteiro como se ele estivesse dividido por uma escala

Ou seja, o padrão de bits armazenado não "tem vírgula" fisicamente.

Quem coloca a vírgula é o formato.

A fórmula é:

$$\text{valor real} = \text{inteiro armazenado} / \text{escala}$$

E a escala vem da quantidade de bits fracionários.

Se o formato tem F bits depois do ponto:

$$\text{escala} = 2^F$$

Então:

$$\text{valor real} = \text{inteiro armazenado} / 2^F$$

4.3.1 Por que a escala é 2^F ?

Porque cada bit fracionário aumenta a quantidade de "pedaços" em que uma unidade é dividida.

Veja:

Bits fracionários	Quantos pedaços cabem em 1 unidade?	Escala
1	2 pedaços	2
2	4 pedaços	4
3	8 pedaços	8
4	16 pedaços	16

Se você tem 4 bits fracionários:

$$F = 4$$
$$\text{escala} = 2^4 = 16$$

Isso significa:

$$16 \text{ unidades armazenadas} = 1 \text{ unidade real}$$

Ou, dito de outro jeito:

$$\text{o número guardado está "16 vezes maior" do que o valor real}$$

Por isso, para ler o valor real, você divide por 16.

4.3.2 Exemplo de leitura

O padrão:

00110100

Como inteiro sem sinal:

00110100_2 = 52

Valor real:

52 / 16 = 3,25

Mesmo resultado.

Então existem dois jeitos equivalentes de ler:

0011,0100 -> 3 + 0,25 = 3,25

ou:

00110100 = 52
52 / 16 = 3,25

Os dois estão certos.

4.3.3 Exemplo de armazenamento

Agora faça o caminho contrário.

Queremos armazenar:

2,5 em Q4.4

Como Q4.4 tem 4 bits fracionários:

escala = 2^4 = 16

Para armazenar, multiplique o valor real pela escala:

inteiro armazenado = valor real * escala
inteiro armazenado = 2,5 * 16
inteiro armazenado = 40

Agora converta 40 para binário:

`40_10 = 00101000_2`

Então:

`2,5 em Q4.4 -> 00101000`

Conferindo:

`00101000_2 = 40`
`40 / 16 = 2,5`

4.3.4 Regra prática

Para ler ponto fixo:

`bits -> inteiro -> divide pela escala`

Para armazenar ponto fixo:

`valor real -> multiplica pela escala -> inteiro -> bits`

Resumo:

Operação	O que fazer
Ler bits em ponto fixo	dividir por <code>2^F</code>
Guardar valor em ponto fixo	multiplicar por <code>2^F</code>

4.4 Formato Q

Ponto fixo costuma ser escrito como formato `Q`.

Exemplos:

`Q4.4 -> 4 bits inteiros e 4 bits fracionários`
`Q3.5 -> 3 bits inteiros e 5 bits fracionários`
`Q1.7 -> 1 bit inteiro e 7 bits fracionários`

Em algumas convenções, para números com sinal, o bit de sinal pode estar incluído na parte inteira.

Então, em prova, leia a definição dada.

Para nosso estudo, use a ideia principal:

Qm.n -> m bits antes do ponto e n bits depois do ponto

4.5 Resolução

Resolução é o menor passo que o formato consegue representar.

Em ponto fixo, a resolução é o valor real de 1 no inteiro armazenado.

Exemplo em Q4.4:

escala = 16

Então:

1 unidade armazenada = 1/16 unidade real
1 unidade armazenada = 0,0625

Logo:

resolução = 0,0625

Isso significa que os valores representáveis andam de 0,0625 em 0,0625.

Exemplo:

Inteiro armazenado	Valor real em Q4.4
0	$0 / 16 = 0$
1	$1 / 16 = 0,0625$
2	$2 / 16 = 0,125$
3	$3 / 16 = 0,1875$
4	$4 / 16 = 0,25$
5	$5 / 16 = 0,3125$

Então, em Q4.4, você consegue representar:

0
0,0625
0,125
0,1875
0,25
...

Mas não consegue representar exatamente:

0,1
0,2
0,3

porque eles não caem exatamente nos passos de 0,0625.

4.5.1 Fórmula da resolução

Se há F bits fracionários:

$$\text{resolução} = 1 / 2^F$$

Como:

$$\text{escala} = 2^F$$

também podemos dizer:

$$\text{resolução} = 1 / \text{escala}$$

Ou seja:

escala e resolução são inversas

Exemplos:

Bits fracionários F	Escala 2^F	Resolução $1/2^F$
1	2	$1/2 = 0,5$
2	4	$1/4 = 0,25$
3	8	$1/8 = 0,125$
4	16	$1/16 = 0,0625$
8	256	$1/256 = 0,00390625$

4.5.2 Diferença entre escala e resolução

Essa é a parte que costuma confundir.

Escala:

é o número pelo qual você divide o inteiro armazenado

Resolução:

é o tamanho do menor passo real representável

Exemplo em Q4.4:

escala = 16
resolução = $1/16 = 0,0625$

Se o inteiro armazenado aumenta de 1:

40 -> 41

o valor real aumenta só:

$1/16 = 0,0625$

Veja:

Inteiro armazenado	Valor real
40	$40/16 = 2,5$
41	$41/16 = 2,5625$
42	$42/16 = 2,625$

Então:

escala = divisor
resolução = passo

4.5.3 Troca entre faixa e precisão

Quanto mais bits fracionários:

maior precisão fracionária
menor faixa para a parte inteira, se o total de bits for fixo

Exemplo com 8 bits sem sinal:

Formato	Bits inteiros	Bits fracionários	Escala	Resolução	Maior valor
Q6.2	6	2	4	0,25	63,75
Q4.4	4	4	16	0,0625	15,9375
Q2.6	2	6	64	0,015625	3,984375

Repare:

Q6.2 -> alcança valores maiores, mas tem passos maiores
Q4.4 -> meio termo
Q2.6 -> passos menores, mas alcança valores menores

Essa troca é central em ponto fixo.

5. Ponto Fixo Com Sinal

5.1 Usando complemento de 2 com escala

Podemos usar complemento de 2 também em ponto fixo.

A ideia é:

interprete o padrão como inteiro com sinal
depois divida pela escala

Exemplo com 8 bits e 4 bits fracionários:

1111 0000

Como inteiro em complemento de 2:

11110000 = -16

Escala:

$2^4 = 16$

Valor real:

$-16 / 16 = -1,0$

Então:

1111,0000 representa -1,0

5.2 Outro exemplo

Formato:

8 bits totais
4 bits fracionários
complemento de 2

Padrão:

1110 1000

Como inteiro com sinal:

11101000

Para achar o valor:

inverte: 00010111
soma 1: 00011000
00011000 = 24

Logo:

11101000 = -24

Divida pela escala:

-24 / 16 = -1,5

Valor:

-1,5

6. Operações Em Ponto Fixo

6.1 Soma e subtração

Se os dois números usam o mesmo formato, soma e subtração são simples.

Exemplo em Q4.4:

1,5 + 2,25

Codifique:

$$1,5 * 16 = 24 \rightarrow 00011000$$
$$2,25 * 16 = 36 \rightarrow 00100100$$

Some os inteiros:

$$\begin{array}{r} 00011000 \\ + 00100100 \\ \hline 00111100 \end{array}$$

Interprete com escala 16:

$$00111100 = 60$$
$$60 / 16 = 3,75$$

Resultado:

$$1,5 + 2,25 = 3,75$$

6.2 Multiplicação

Na multiplicação, o cuidado aumenta.

Se dois valores têm F bits fracionários, o produto cru fica com $2F$ bits fracionários.

Exemplo simples:

$$1,5 * 2,0 = 3,0$$

Em Q4.4:

$$1,5 \rightarrow 24$$
$$2,0 \rightarrow 32$$

Produto inteiro:

$$24 * 32 = 768$$

Mas esse produto está escalado por:

$$16 * 16 = 256$$

Valor real:

$$768 / 256 = 3,0$$

Para voltar para Q4.4, divide por 16:

$$768 / 16 = 48$$

E:

$$48 / 16 = 3,0$$

Em hardware, dividir por 16 pode ser feito por deslocamento:

shift right 4

Conexão com a Aula 19:

shifter ajuda a ajustar escala

6.3 Overflow em ponto fixo

Ponto fixo também pode ter overflow.

Se o formato tem poucos bits para a parte inteira, valores grandes não cabem.

Exemplo em Q4.4 sem sinal:

maior valor = 1111,1111

Valor:

$$15 + 15/16 = 15,9375$$

Então:

20,0 não cabe nesse formato

7. Ponto Flutuante

7.1 Ideia principal

Ponto flutuante é parecido com notação científica.

Em decimal, escrevemos:

$$602000000000000000000000 = 6,02 \cdot 10^{23}$$

Em binário, a ideia é:

```
valor = sinal * significando * 2^expoente
```

O ponto "flutua" porque ele pode se mover conforme o expoente.

Por isso o formato consegue representar números:

muito grandes
 muito pequenos

7.2 Campos principais

Um número em ponto flutuante costuma ter:

```
sinal
expoente
fração/mantissa/significando
```

Modelo conceitual:

S	Expoente	Fração
---	----------	--------

Significado:

Campo	Função
Sinal	Define positivo ou negativo
Expoente	Define a escala/potência de 2
Fração	Define os bits significativos do número

7.3 Normalização

Em binário, um número normalizado costuma ficar na forma:

1.xxxxx * 2^e

Exemplo:

1101,01_2

Mova o ponto até ficar um 1 antes dele:

1101,01 = 1,10101 * 2^3

Por quê 2^3 ?

Porque o ponto andou 3 posições para a esquerda:

1101,01
1,10101

Logo:

1101,01_2 = 1,10101_2 * 2^3

7.4 Bit implícito

Em muitos formatos, como IEEE 754 normalizado, o 1 antes do ponto não é armazenado.

Ele é assumido.

Exemplo:

1,10101 * 2^3

O formato pode armazenar apenas:

10101...

O 1 inicial fica implícito.

Isso economiza um bit de precisão.

8. Expoente Com Viés

8.1 Por que usar viés?

O expoente pode ser positivo ou negativo.

Exemplos:

$$1,101 * 2^3$$
$$1,101 * 2^{-4}$$

Para evitar armazenar o expoente diretamente em complemento de 2, muitos formatos usam viés, também chamado de bias.

Regra:

$$\text{expoente armazenado} = \text{expoente real} + \text{bias}$$

No IEEE 754 precisão simples:

$$\text{expoente tem 8 bits}$$
$$\text{bias} = 127$$

Então:

$$\text{expoente real} = \text{expoente armazenado} - 127$$

8.2 Exemplo conceitual

Se o expoente real é:

3

No formato com bias 127:

$$\text{expoente armazenado} = 3 + 127$$
$$\text{expoente armazenado} = 130$$

Em binário:

$$130 = 10000010$$

Então o expoente armazenado seria:

10000010

9. Exemplo De Ponto Flutuante Conceitual

Converta o valor:

13,25₁₀

9.1 Converter para binário

Parte inteira:

13 = 1101₂

Parte fracionária:

0,25 = 0,01₂

Então:

13,25₁₀ = 1101,01₂

9.2 Normalizar

1101,01₂ = 1,10101₂ * 2³

Logo:

```
sinal = 0
expoente real = 3
significando = 1,10101
fração armazenada = 10101...
```

Se fosse IEEE 754 precisão simples:

```
bias = 127
expoente armazenado = 3 + 127 = 130
130 = 10000010
```

Representação conceitual:

```
S = 0
E = 10000010
F = 101010000000000000000000
```

Não precisa decorar todos os bits para a prova se ela não pedir IEEE 754 completo.

Mas precisa entender:

sinal diz positivo/negativo
expoente diz escala
fração diz os bits importantes do número

10. Precisão, Arredondamento E Erro

10.1 Ponto flutuante não representa tudo exatamente

O número de bits da fração é limitado.

Então alguns valores precisam ser aproximados.

Exemplo:

0,1 decimal

Em binário:

0,00011001100110011...

Como não há infinitos bits disponíveis, o computador corta ou arredonda.

Então o valor armazenado é próximo de 0,1, mas não exatamente 0,1.

10.2 Consequência prática

Pode acontecer:

0,1 + 0,2 não dar exatamente 0,3

Isso não significa que o processador "errou" a matemática.

Significa:

os valores foram representados aproximadamente
a operação usou esses valores aproximados
o resultado também foi arredondado

10.3 Faixa versus precisão

Ponto flutuante troca precisão por faixa.

Ele consegue representar números muito grandes e muito pequenos, mas não representa todos os números reais.

Pense assim:

ponto fixo -> espaçamento constante entre valores
ponto flutuante -> espaçamento relativo, muda com o expoente

Com expoente grande, a distância entre dois números representáveis também fica maior.

11. Ponto Fixo Versus Ponto Flutuante

Característica	Ponto fixo	Ponto flutuante
Posição do ponto	Fixa	Varia com o expoente
Hardware	Mais simples	Mais complexo
Faixa de valores	Menor	Maior
Precisão	Uniforme na escala escolhida	Varia conforme o expoente
Erros	Quantização pela escala	Arredondamento e precisão limitada
Uso comum	Controle, DSP simples, sistemas embarcados	Cálculo científico, gráficos, IA, simulações

Resumo:

ponto fixo é como guardar um inteiro com escala combinada
ponto flutuante é como guardar notação científica em base 2

12. Como Questões Costumam Cobrar

12.1 Conversão de fração binária para decimal

Pergunta típica:

Quanto vale $10,101_2$ em decimal?

Resolução:

$10_2 = 2$
 $0,101_2 = 1/2 + 1/8 = 0,625$
total = 2,625

12.2 Interpretação de ponto fixo

Pergunta típica:

O padrão 00110100 em Q4.4 representa qual valor?

Resolução:

$00110100 = 52$
 $52 / 16 = 3,25$

12.3 Formato Q

Pergunta típica:

Em Q3.5, qual é a resolução?

Resolução:

$F = 5$
 $\text{resolução} = 1/2^5 = 1/32 = 0,03125$

12.4 Normalização

Pergunta típica:

Normalize 10110,01_2.

Resolução:

$10110,01_2 = 1,011001_2 * 2^4$

12.5 Campos do ponto flutuante

Pergunta típica:

Quais campos formam um número em ponto flutuante?

Resposta:

sinal
expoente
fração/mantissa/significando

12.6 Erro de precisão

Pergunta típica:

Por que 0,1 pode não ser representado exatamente em binário?

Resposta:

porque 0,1 decimal vira uma dízima periódica em binário
e o formato tem quantidade limitada de bits

13. Erros Comuns

13.1 Achar que o ponto fica armazenado em ponto fixo

Em ponto fixo, os bits armazenados são apenas bits.

O ponto é uma convenção do formato.

00110100

Pode significar 52 como inteiro sem sinal.

Mas em Q4.4 significa:

3,25

O padrão é o mesmo.

A interpretação muda.

13.2 Esquecer a escala

Se há 4 bits fracionários:

escala = 16

Não interprete o inteiro cru como o valor real.

13.3 Achar que todo shift à esquerda é multiplicação segura por 2

Em largura fixa, bits podem ser descartados.

Então:

shift left pode causar overflow/perda de informação

13.4 Confundir mantissa com número inteiro comum

Em ponto flutuante normalizado, a fração representa os bits depois do 1, .

Exemplo:

$1,10101 \cdot 2^3$

A fração armazenada é:

10101...

13.5 Achar que ponto flutuante é sempre mais preciso

Ponto flutuante tem faixa maior, mas isso não significa precisão infinita.

Ele aproxima muitos valores.

14. Exemplos Resolvidos

14.1 Converter binário fracionário

Converta:

11,011₂

Parte inteira:

$11_2 = 3$

Parte fracionária:

$0,011_2 = 0/2 + 1/4 + 1/8$
 $0,011_2 = 0,25 + 0,125$
 $0,011_2 = 0,375$

Resultado:

$$11,011_2 = 3,375_{10}$$

14.2 Converter decimal fracionário

Converta:

$$6,75_{10}$$

Parte inteira:

$$6 = 110_2$$

Parte fracionária:

$$\begin{aligned} 0,75 * 2 &= 1,5 \rightarrow \text{bit } 1, \text{ resta } 0,5 \\ 0,5 * 2 &= 1,0 \rightarrow \text{bit } 1, \text{ resta } 0 \end{aligned}$$

Logo:

$$0,75 = 0,11_2$$

Resultado:

$$6,75_{10} = 110,11_2$$

14.3 Interpretar Q4.4

Padrão:

$$0101 \ 1000$$

Em inteiro:

$$01011000_2 = 88$$

Escala:

$$2^4 = 16$$

Valor:

$$88 / 16 = 5,5$$

14.4 Interpretar Q4.4 com sinal

Padrão:

1111 1000

Como inteiro com sinal em complemento de 2:

$$11111000 = -8$$

Escala:

16

Valor:

$$-8 / 16 = -0,5$$

14.5 Normalizar número binário

Normalize:

101,11₂

Mova o ponto:

$$101,11 = 1,0111 * 2^2$$

Resultado:

$$1,0111_2 * 2^2$$

14.6 Valor de um formato flutuante conceitual

Considere:

sinal = 0
significando = 1,101

expoente real = 3

Valor:

$+1,101_2 \cdot 2^3$

Primeiro:

$1,101_2 = 1 + 1/2 + 0/4 + 1/8$
 $1,101_2 = 1,625$

Agora:

$1,625 \cdot 2^3 = 1,625 \cdot 8 = 13$

Valor:

13

15. Exercícios

15.1 Conceituais

1. O que significa dizer que uma representação é de ponto fixo?
2. Em ponto fixo, o ponto binário é armazenado junto com os bits?
3. Qual é a relação entre bits fracionários e escala?
4. O que significa resolução em ponto fixo?
5. Quais são os três campos principais de um número em ponto flutuante?
6. O que é normalização em ponto flutuante?
7. Por que ponto flutuante pode ter erro de arredondamento?
8. Qual é a diferença principal entre ponto fixo e ponto flutuante?

15.2 Conversões e interpretação

9. Converta $10,1_2$ para decimal.
10. Converta $101,01_2$ para decimal.
11. Converta $0,625_{10}$ para binário.

12. Converta `5,25_10` para binário.
13. Interprete `0010 1000` em Q4.4 sem sinal.
14. Interprete `0111 0000` em Q4.4 sem sinal.
15. Interprete `1111 0000` em Q4.4 com sinal em complemento de 2.
16. Qual é a resolução de um formato com 6 bits fracionários?

15.3 Ponto flutuante

17. Normalize `1101,1_2`.
18. Normalize `0,011_2`.
19. Em IEEE 754 precisão simples, se o expoente real é `5`, qual é o expoente armazenado em decimal?
20. O que significa o bit implícito em ponto flutuante normalizado?
21. Um número flutuante conceitual tem `sinal=1`, `significando=1,01_2` e `expoente=2`. Qual é o valor decimal?
22. Explique por que `0,1_10` não é representado exatamente em binário finito.

15.4 Questões mais parecidas com prova

23. Um sistema usa Q3.5. Quantos bits fracionários existem e qual é a resolução?
24. Em Q4.4, o valor real `2,5` deve ser armazenado como qual inteiro bruto?
25. Em Q4.4 sem sinal, o padrão `1111 1111` representa qual valor?
26. Qual representação é mais adequada para hardware simples e valores com faixa limitada: ponto fixo ou ponto flutuante?
27. Qual representação é mais adequada para cobrir uma faixa muito grande de valores: ponto fixo ou ponto flutuante?
28. O padrão de bits `0100 0000` pode representar valores diferentes dependendo do formato? Explique.

16. Gabarito

1. Significa que a posição do ponto binário é fixa por convenção do formato.
2. Não. O ponto não é armazenado; ele é assumido pelo formato.
3. Se há `F` bits fracionários, a escala é `2^F`.
4. É o menor passo entre dois valores representáveis. Com `F` bits fracionários, é `1/2^F`.
5. Sinal, expoente e fração/mantissa/significando.
6. É escrever o número na forma `1.xxxxx * 2^e`.
7. Porque há quantidade limitada de bits para representar a fração.
8. Ponto fixo tem ponto em posição fixa; ponto flutuante usa expoente para variar a escala.

9. $10,1_2 = 2 + 0,5 = 2,5$.
10. $101,01_2 = 5 + 0,25 = 5,25$.
11. $0,625_{10} = 0,101_2$.
12. $5,25_{10} = 101,01_2$.
13. $00101000_2 = 40$; $40/16 = 2,5$.
14. $01110000_2 = 112$; $112/16 = 7,0$.
15. $11110000_2 = -16$ em complemento de 2; $-16/16 = -1,0$.
16. $1/2^6 = 1/64 = 0,015625$.
17. $1101,1_2 = 1,1011_2 * 2^3$.
18. $0,011_2 = 1,1_2 * 2^{-2}$.
19. Bias 127: $5 + 127 = 132$.
20. É o 1 antes do ponto em números normalizados. Ele não é armazenado, mas é assumido.
21. $sinal=1$ indica negativo. $1,01_2 = 1,25$. $1,25 * 2^2 = 5$. Valor: -5 .
22. Porque $0,1_{10}$ vira uma dízima periódica em binário, exigindo infinitos bits.
23. Existem 5 bits fracionários. Resolução: $1/32 = 0,03125$.
24. Em Q4.4, escala 16. Inteiro bruto: $2,5 * 16 = 40$, ou 00101000 .
25. $11111111_2 = 255$; $255/16 = 15,9375$.
26. Ponto fixo.
27. Ponto flutuante.
28. Sim. Como inteiro sem sinal, 01000000 é 64. Em Q4.4, é $64/16 = 4,0$. O mesmo padrão de bits pode ter interpretações diferentes.

17. O Que Memorizar

Pesos depois do ponto binário:

$$2^{-1} = 0,5$$

$$2^{-2} = 0,25$$

$$2^{-3} = 0,125$$

$$2^{-4} = 0,0625$$

Ponto fixo:

$$\text{valor real} = \text{inteiro armazenado} / 2^F$$

F = quantidade de bits fracionários

$$\text{escala} = 2^F$$

$$\text{resolução} = 1 / 2^F$$

escala = divisor usado para ler o inteiro armazenado
resolução = menor passo real representável
resolução = $1 / \text{escala}$

Q_{m.n}:
m bits antes do ponto
n bits depois do ponto

Ponto flutuante:
valor = sinal * significando * 2^{expoente}

Normalizado em binário:
 $1.\text{xxxxx} * 2^e$

IEEE 754 precisão simples:
sinal = 1 bit
expoente = 8 bits
fração = 23 bits
bias = 127

Ponto fixo -> simples, escala fixa
Ponto flutuante -> faixa maior, arredondamento e precisão limitada

18. Plano De Estudo Para Esta Aula

Sugestão:

1. Ler seções 2 e 3 para dominar frações em binário.
2. Memorizar os pesos 0,5; 0,25; 0,125; 0,0625.
3. Fazer 6 conversões binário fracionário -> decimal.
4. Fazer 4 conversões decimal fracionário -> binário.
5. Ler ponto fixo e refazer exemplos Q4.4.
6. Ler ponto flutuante focando em normalização.
7. Resolver exercícios 9 a 28.

Se tiver pouco tempo, priorize:

9, 10, 12, 13, 15, 17, 18, 21, 24 e 28

Esses exercícios pegam:

fração binária
escala em ponto fixo

complemento de 2 com escala
normalização
interpretação dependente do formato

19. Conexão Com A Próxima Aula

Na próxima aula, vamos voltar para blocos aritméticos e estudar como o hardware faz subtração, comparação, deslocamento e seleção de operações:

subtrator
comparador
shifter
ULA

Isso conecta ponto fixo/flutuante com o restante da aritmética digital:

representação numérica
operações em hardware
controle da ULA
interpretação de overflow, sinal e escala

Depois, a sequência segue primeiro para somadores, depois para ULA e então para a parte física:

somadores
subtrator, comparador, shifter e ULA
física básica
transistores MOS
CMOS combinacional